RuSMT: One Executable Reference Semantics as Both Conformance Oracle and Solver-Directed Test Synthesizer

Mehrad Haghshenas Meng Xu
University of Waterloo, Waterloo, ON, Canada
m3haghsh@uwaterloo.ca meng.xu.cs@uwaterloo.ca

Abstract

A language's prose specification, reference implementation, and test generator are three artifacts that must agree, but in practice they drift apart. We present RuSMT, a Rust-embedded DSL in which a single *direct-style executable reference semantics* plays two roles at once: executed as ordinary Rust it is a concrete conformance oracle, and compiled automatically to SMT it is the source for solver-directed *bounded synthesis* of test programs. An author writes the operational semantics once over a standard library of types that mirror Z3's theories, marks program points of interest with `Path::fresh()`, and RuSMT lifts the interpreter to a Z3 query whose models are inputs that reach those path targets. Decoded back to source, each witness is replayed through the same interpreter and then run against an independent commercial implementation, where a divergence is a conformance bug. We make the design's soundness explicit: a per-construct contract requires every standard-library primitive to agree with its Z3 formula, a guard discipline reconciles the inputs on which they would diverge, and synthesis soundness is enforced operationally by replay. We claim neither a new synthesis algorithm nor general completeness—only *bounded completeness*: for every marker, every program of depth at most k that reaches it is representable in our SMT encoding, subject to Z3 returning a model rather than timing out.The contribution is therefore concrete: (i) a restricted Rust DSL in which a language designer authors a single executable reference semantics and obtains from that one artifact both a runnable conformance oracle and a solver-directed test generator; (ii) the soundness of the DSL→IR→SMT compilation pipeline, anchored on a per-construct contract that pins each Rust primitive to its emitted Z3 formula and documents, where target-language semantics diverge from Z3's, how each divergence is reconciled; and (iii) two case studies—an IMP/WHILE interpreter and a TOML 1.1.0 parser—that exemplify the authoring pattern across a small canonical language and a real-world specification. The IMP loop closes end-to-end—synthesis, witness, replay—while the TOML parser reaches the solver frontier: deeply recursive queries combining bit-vectors, strings, arrays, and quantifiers.

Keywords: executable semantics · conformance testing · bounded program synthesis · SMT · Z3 · solver-aided programming · Rust DSL.

1 Introduction

A language ecosystem typically carries three artifacts that are supposed to say the same thing: a prose *specification*, a *reference implementation*, and a *test generator* for conformance. They are written and maintained separately, so they drift. Random fuzzing covers the bulk of the input space but tends to miss the narrow error and edge conditions a specification cares about most. On the other hand, a hand-written test suite forces its author to do two things at once: decide which conditions are interesting and craft inputs that exercise them. RuSMT keeps the author's judgment about what is interesting—now expressed as markers in the semantics itself—and hands input generation to the solver.

RuSMT collapses two of these three artifacts into one. The author writes the reference semantics *once*, in a restricted dialect of Rust embedded in the host language, using a standard library whose types denote Z3's mathematical theories. That single source is given two interpretations (Fig. 1): run as ordinary Rust it is the concrete conformance *oracle*; compiled—automatically, with no hand-written SMT-LIB—it is the source of a solver query that *synthesizes* programs reaching designated points. The author marks a point of interest by writing `Path::fresh()`; RuSMT asks Z3 for an input that drives execution to that marker, decodes the model back into runnable source, replays it through the same interpreter, and hands the witness to an independent implementation. A divergence is a conformance bug. The barrier this lowers is concrete: *no separate relational interpreter, no hand-authored constrained Horn clauses or SMT-LIB, and no separate generator.*

Research question. *Can a single direct-style executable reference semantics, written in a familiar host language, serve simultaneously as a concrete conformance oracle and as the source for solver-directed bounded synthesis of test programs—without rewriting the semantics as a relational program, a CHC encoding, a rewrite system, or a hand-staged synthesis grammar?*

What we do *not* claim. The core mechanism—compile an executable interpreter to a symbolic form and search for inputs—is not new. Rosette [22, 23] does this with Racket as host, letting one program serve as both runtime and symbolic IR. SemGuS [13] does it relationally, asking the user to re-author the semantics as CHCs. Futamura's projections [9] provide the theoretical foundation, recently operationalised for symbolic interpreters by Wei & Rompf [25]: a (symbolic) partial evaluator turns an interpreter into a per-program constraint system. We claim no new theorem and no novel algorithm, and we do not claim Rust-as-host is itself a contribution.

Contributions. We frame RuSMT as a tool/experience contribution:

1. **An integrated, direct-style, Rust-native pipeline** that lifts an ordinary recursive interpreter to marker-directed bounded synthesis, with two interchangeable solver backends (an SMT-LIB text backend driving a Z3 subprocess, and an in-process backend that builds Z3 terms via `z3-sys`) that can be run together to cross-check (§4).
2. **A documented per-construct soundness contract** for the standard library—every primitive must equal its Z3 formula on every input—together with the guard discipline that reconciles divergences, and an honest account of how the contract is actually validated (§5).
3. **An explicit treatment of synthesis soundness versus completeness**, with soundness enforced operationally by replay and completeness deliberately *not* claimed beyond a structural bound; and the single-free-input/fixed-context methodological principle that makes witnesses replayable (§§5—6).
4. **Two case studies and an empirical characterization of the solver frontier** (§§7—8): IMP closes end-to-end, while a full TOML 1.1.0 parser reaches the limits of the solver on deeply recursive, multi-theory, quantified queries. We present the frontier as a general finding about solver-aided synthesis from recursive interpreters, *illustrated by* the case studies rather than caused by them.

A motivating fragment. Listing 1 shows the division case of IMP's arithmetic evaluator. The interpreter is ordinary recursive Rust. The only synthesis-specific token is `Path::fresh()`: it marks the division-by-zero branch as a point we want a test to reach. From this single source, RuSMT synthesizes the program `A := (0 / 0)` (§8); replayed through the very same `eval_aexp`, it reaches the marker.

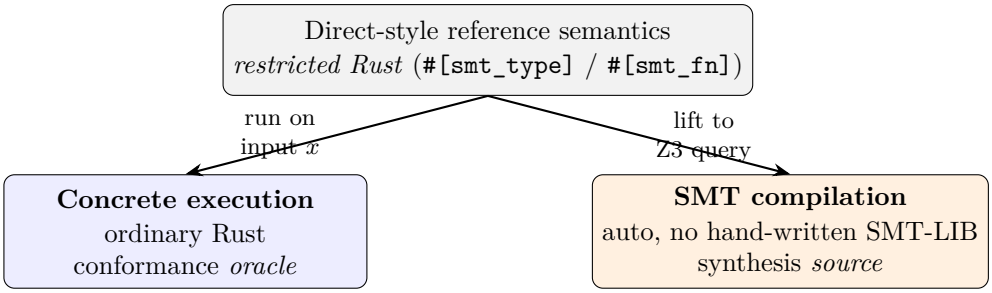

Figure 1: One source of truth, two interpretations. The same restricted-Rust semantics runs concretely (the oracle) and compiles to a Z3 query (the synthesis source).

```
Aexp::Div(l, r) => match eval_aexp(l.reveal(), s) {
    ArithmeticResult::Err(e) => ArithmeticResult::Err(e),
    ArithmeticResult::Ok(new_l) => match eval_aexp(r.reveal(), s) {
        ArithmeticResult::Err(e) => ArithmeticResult::Err(e),
        ArithmeticResult::Ok(new_r) => {
            if *new_r.eq(I64::from(0)) {
                ArithmeticResult::Err(Path::fresh())   // marker: division by zero
            } else {
                ArithmeticResult::Ok(new_l.bv_div(new_r))
            }
        }
    },
},
```

Listing 1: The division case of IMP's evaluator (from `lang/src/imp/mod.rs`).

Forward map. §2 fixes background. §3 describes the DSL. §4 describes the compilation pipeline. §5 is the soundness story. §6 is bounded synthesis. §7 presents the two case studies and §8 the evaluation. §9 positions the work, §10 states limitations, and §11 concludes.

2 Background

SMT and Z3. A satisfiability-modulo-theories (SMT) solver decides logical formulas over background theories—booleans, integers and reals, fixed-width bit-vectors, IEEE floats, strings, sequences, arrays, and algebraic datatypes. We target Z3 [7] through the SMT-LIB 2.6 input language [2]. A query is *sat* (a model exists), *unsat* (none does and a proof is found), or *unknown*. The *unknown* verdict is fundamental: SMT in full generality is undecidable. Decidability can be lost in any of several independent ways—individual theories that are themselves undecidable (e.g., nonlinear integer arithmetic), combinations of theories that violate the preconditions of standard combination procedures, quantifier alternation, and recursive function definitions. Z3 handles quantifiers via heuristic instantiation (e-matching and model-based quantifier instantiation, MBQI) that is necessarily incomplete; on decidable fragments, the solver may still return *unknown* when its resource budget is exhausted.

Executable semantics. An interpreter that runs the language it specifies is an *executable* semantics. Tools such as the K framework [20], PLT Redex [8], and Ott/Lem [21, 16] let one write semantics as rewrite or inference rules and derive interpreters, test oracles, and proof-assistant definitions directly from the semantics. The premise is straightforward: when all implementations share one semantic source, specification and implementation agree by construction. Using this approach, RuSMT takes the semantics to be an *ordinary recursive function* in the host language and derives one tool—marker-directed test synthesis—from it.

Conformance testing and synthesis. Conformance testing checks an implementation against a specification. Generators range from hand-written suites to grammar- and coverage-driven fuzzers such as Csmith [27]. Program *synthesis* searches a space of programs for one meeting a specification; syntax-guided synthesis (SyGuS) [1] fixes a grammar and a logical spec, and semantics-guided synthesis (SemGuS) [13] adds user-supplied semantics as constrained Horn clauses (CHCs) [3]. RuSMT synthesizes *inputs* (programs in the object language) that reach a path condition in a *given* interpreter; the interpreter is written as a function, not as CHCs or a grammar.

3 The RuSMT DSL

A RuSMT specification is a Rust program written under restrictions that make it translatable to SMT. The restrictions are intentionally severe: programs use immutable `let` bindings, `if` (with a mandatory `else`), `struct`/`enum`/`match`, and (mutually) recursive function calls. There are *no loops*, *no mutation*, and *no references*; recursion and quantifiers are the only iterative constructs. Two attribute macros from the `rusmt-smt-remark` crate enforce the subset. `#[smt_type]` accepts only `struct`s and `enum`s and injects a `Copy`-everything derive set, so every value is `Copy` (there are no references to alias). `#[smt_fn]` rejects `const`/`async`/`unsafe`/variadic functions and method receivers. The body restriction—only `let` and a trailing expression, no loop or assignment forms—is enforced when the function parser rejects every syntactic form it does not explicitly handle.

Standard library: the theory vocabulary. The DSL's types denote Z3 sorts: `Boolean` (→ `Bool`); `Integer` and `Real` (unbounded `Int` and `Real`, backed by big integers and rationals); the bit-vectors `I32`, `I64`, `U32`, and `U64` (→ `(_ BitVec 32)` or `(_ BitVec 64)`, the signed/unsigned distinction being a DSL-API interpretation of the same Z3 sort); `F32` and `F64` (IEEE `FloatingPoint`); `String` (the Strings theory); `Seq<T>`, `Set<T>`, and `Array<K,V>`; and two special types, `Cloak<T>` and `Path`, described below. Crucially, the standard library represents *Z3's* mathematical theories—not Rust's native semantics and not any target language's semantics. Target-language behavior is encoded *in the interpreter* using these primitives as building blocks (§5).

Self-referential ADTs via `Cloak<T>`. Algebraic syntax trees are naturally self-referential, e.g. `Add(Aexp, Aexp)`, but Rust forbids infinitely sized types. `Cloak<T>` is a frontend-only wrapper (analogous to `Box<T>`) that lets the author write `Add(Cloak<Aexp>, Cloak<Aexp>)`; values are wrapped with `Cloak::shield` and read with `.reveal()`. The intermediate representation (IR) *erases* `Cloak` entirely: every `Cloak<T>` field lowers to `T`, `shield`/`reveal` lower to identity, and the SMT output never mentions `Cloak` (Z3 supports recursive datatypes natively). The only proviso is well-foundedness: a `Cloak<T>` must co-occur with at least one non-recursive variant, so the lowered datatype is inhabited.

Markers via `Path`. `Path::fresh()` allocates a unique marker identifier at a program point of interest; `Path::merge` unions marker sets. A `Path` value is a set of integer ids, represented in Z3 as `(Array Int Bool)` (a characteristic function). Markers are *neutral tags*: they flag where the author wants test coverage—error cases, edge conditions—and are not necessarily bugs. Each marker (or a merged, disjoint set of ids) becomes one synthesis target (§6).

Quantifiers. The DSL provides `forall!`, `exists!`, and `choose!` in a *bounded* form over a collection: `forall!(x in xs => p)` and `choose!(x in xs => p)`. `choose!` is Hilbert choice—concretely it returns the first witness satisfying the predicate; in SMT it is a Skolem function

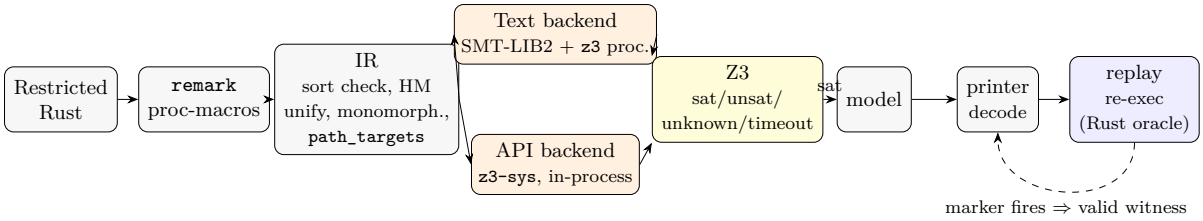

Figure 2: The RuSMT pipeline. Restricted Rust passes through the `remark` macros, is lowered to a sort-checked IR (with Hindley—Milner unification, on-demand monomorphization, and collection of `path_targets`), and is emitted by one of two backends to Z3. A *sat* model is decoded to `.imp` source and replayed through the concrete oracle; if the marker fires, the witness is valid.

constrained by the standard choose axiom. Soundness depends only on the chosen value *satisfying* the predicate, never on *which* value is chosen, so the Rust-vs-SMT difference is immaterial.

4 The compilation pipeline

Figure 2 shows the pipeline. The same annotated Rust source is either compiled by `rustc` and executed, or fed to the `rusmt-smt-derive` transpiler.

Parser and IR. The transpiler parses the restricted Rust into its own AST and lowers it to a sort-checked IR. Type inference is a simplified Hindley—Milner with union-find unification over a fixed lattice of SMT sorts: equivalence classes of type variables are merged on the fly, with an occurs check that rejects cyclic unification. It is "simpler than full HM because it operates on a fixed set of SMT sorts rather than arbitrary polymorphic types." Generics are *monomorphized on demand*: function and type instantiations are cached by their concrete sort vector, and a call site instantiates (and registers) a new monomorph only on a cache miss. (Z3 supports parametric *types* but not parametric *functions*, so functions must be monomorphized; types may be emitted once as a parametric template.) During lowering, each `Path::fresh()` is assigned a unique id and recorded in `IRContext::path_targets`, a list of id-sets; `Path::merge` records the merged set. Each entry becomes one synthesis query.

Two backends behind one trait. Both backends consume the same IR through a shared `CodeGen` interface (`process` emits the base program; `invoke_backend` runs the solver; `process_path_queries` turns one target into a complete query). Both apply the same Z3 configuration—`smt.auto_config=false` (so the manual settings stick), `smt.mbqi=true`, `smt.ematching=true`—differing only in how they transmit it. The *text* backend renders IR intrinsics to SMT-LIB2, writes a `.smt2` file (kept on disk so it is inspectable), emits the configuration as `set-option` directives at the top of the file, and spawns a `z3` subprocess; it passes Z3 a self-timeout (`-t:`) and additionally guards the subprocess with a watchdog thread, draining stdout and stderr on separate threads to avoid pipe deadlock, and reads the verdict as the first line equal to `sat`/`unsat`/`unknown` *before* consulting the exit code (Z3 exits non-zero for benign reasons such as model generation after `unknown`). The *API* backend builds Z3 datatypes and terms in memory via `z3-sys`, applies the same configuration via `set_global_param`, additionally fixes reproducible random seeds, gives each target a fresh context, and asserts each query through the C API. The two backends are independent implementations of the same semantics and can be run together (`both`) to cross-check; their model *formats* differ (the API backend prints `let`-shared subterms and an `input_0 -> ...` form), which is not a semantic distinction.

5 Soundness

The trusted computing base. The RuSMT specification *is* the trusted base. We *assume* it correct: the manual transcription of a prose specification or standard into the DSL is trusted and is not itself verified. Everything downstream—the SMT compilation, the solver, and the synthesized witnesses—is checked back against this trusted reference *by re-execution*. In the case studies, the concrete parser (source → AST) and the pretty-printer (AST → source), together with Z3 itself, are also part of the trusted base; a bug in any of them is a bug in the results.

The per-construct contract. Soundness rests on a contract for every standard-library primitive:

Definition 1 (Per-construct soundness contract)**.** For every stdlib function f and every concrete input x,

$$\mathit{rust\_impl}(f, x) \;=\; \mathit{z3\_eval}(\mathit{z3\_formula}(f),\, x).$$

That is, evaluating f concretely in Rust must equal evaluating Z3's formula for f on the encoded value. The stdlib denotes Z3's theories; bridging the stdlib to a given target language is the interpreter author's job.

The guard / if-then-else pattern. When a primitive matches the target language, the author uses it directly. When it diverges only on specific inputs, the author *guards* those inputs and falls through to the (Z3-correct) primitive otherwise. Listing 1 is exactly this shape: `bv_div` agrees with Z3 everywhere, so the only special case is a zero divisor, which the guard turns into a marker. Because concrete Rust and Z3 take the *same* branch on the same input, the equality of Definition 1 is preserved through the conditional. The same discipline appears inside the stdlib itself where Rust and Z3 would otherwise disagree—e.g. a bit-vector shift by $\geq$ the width returns 0 to match Z3's `bvshl` rather than Rust's wrapping shift, and `is_negative` guards `NaN` because Z3's `fp.isNegative` is `false` on all NaNs while Rust's `is_sign_negative` is not.

How the contract is validated. Validation is layered, and we are explicit about what each layer does and does not establish. (i) Each stdlib method carries a doc-comment stating the exact Z3 formula it must match—a *public commitment* that anchors review and future audits. (ii) Example-based unit tests pin the Rust side to Z3's documented result for representative inputs (the bit-vector, string, and float suites alone hold dozens of such cases) and have caught real divergences during development. (iii) For selected error-prone intrinsics the equivalence has additionally been checked *universally*, by asserting the negation of Definition 1 and obtaining `unsat` from Z3. What does *not* exist is a checked-in randomized differential harness that fuzzes *rust_impl* against *z3_eval* across all intrinsics, nor is the corpus of universal-equivalence proof scripts a reproducible suite in the tree; both are concrete future work (§10). The cross-backend `both` mode provides an additional, independent consistency check at the whole-query level—it does not establish Definition 1, but it catches whole-pipeline regressions in either backend.

Single free input; fixed context. A subtle but load-bearing methodological point governs *which* inputs are free in the synthesis query.

Principle 1 (Free/fixed agreement)**.** The set of inputs treated as free by the SMT query must coincide with the set of inputs left free by the concrete oracle; everything else must be fixed to the same value on both sides.

The IMP interpreter is run from the *empty* store. Its public entry point therefore takes only the program and fixes the store internally:

```
#[smt_fn]
pub fn eval_com(c: Com) -> EvalResult {
    let s = Array::new();    // context fixed: empty initial store
    eval_com_inner(c, s)     // c is the single free input
}
```

Synthesis then declares exactly one symbolic constant, `input_0 : Com`; the store is not a query variable. Were the store left free, Z3 could choose a starting state that the concrete (empty-store) run never uses, producing a "witness" that does not replay—a soundness hole. TOML's `parse_toml` already takes a single input (a code-point sequence) and builds its cursor and context internally, so it satisfies Principle 1 without a wrapper; the contrast makes the point.

Synthesis soundness via replay.

Observation 1 (Soundness of synthesis). Every witness Z3 returns, when replayed through the concrete interpreter, reaches the targeted marker.

This is enforced *operationally*: the decoded program is re-parsed and re-executed, and only a run that fires the marker is accepted. Replay is necessary because quantified queries can return a *candidate* model—Z3 reports `sat` accompanied by `(:reason-unknown "")`, meaning the model satisfied its instantiation heuristics but is not certified. The pipeline records such a result as `sat` regardless and lets replay be the arbiter. The soundness link holds because the program is replayed on the *same* fixed context (Principle 1) and the stdlib primitives agree with Z3 (Definition 1); identifier and bit-vector decoding (below) do not affect which marker fires.

Completeness: not claimed. We do *not* claim completeness in general. Even when an input reaching a marker exists, Z3 may return `unknown` or time out: the encoded formula can fall outside what the solver decides within its resource limits. At best we have *bounded* completeness—relative to a structural/fuel bound (§6)—and even that is contingent on the solver. We state this plainly rather than gesture at a guarantee we do not have.

6 Bounded synthesis

Markers to queries. For a target id i, the query declares one constant per top-level parameter (`input_0`, ...), applies the entry function to them, and asserts that the result is the error value whose carried `Path` set contains i. A `Path` is `(Array Int Bool)`; `Path::fresh(`i`)` is `(store ((as const (Array Int Bool)) false)` i `true)` and membership is a `select`. For an enum return type the assertion is guarded by the constructor test. Listing 2 is the actual query RuSMT emits for IMP's division-by-zero marker.

```
(declare-const input_0 Com)
(assert (and (is-EvalResult_Err (eval_com input_0))
             (select (field_EvalResult_Err_1_ (eval_com input_0)) 0)))
(check-sat)
(get-info :reason-unknown)
(get-model)
```

Listing 2: The synthesis query for one IMP marker (text backend). "Find an `input_0 : Com` whose evaluation is `EvalResult::Err` carrying marker 0."

Bounded recursion by unrolling. A recursive interpreter is not a finite formula. RuSMT offers two routes, selected by a `k=N` flag. With `N=0` (default) recursive strongly-connected components (SCCs) are emitted as Z3 `define-funs-rec` and the solver handles recursion

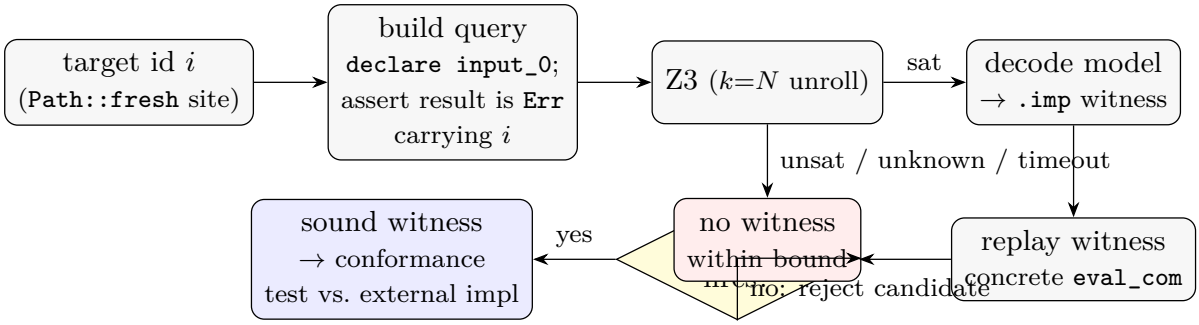

Figure 3: The synthesis/replay loop for one marker target. Replay is what distinguishes a genuine witness from a candidate model.

natively. With `N ≥ 1` every recursive SCC is unrolled into $N{+}1$ non-recursive, depth-indexed copies $f_0, \ldots, f_N$: copy f_d calls f_{d-1} for in-SCC callees, f_0 is a terminator (the first nullary enum constructor, e.g. `NoMatch`/`None`, or a null sentinel), and external callers route through f_N via an alias. SCCs are found by Kosaraju's algorithm on a call graph that follows `let`-bound calls; a singleton without a self-loop is treated as non-recursive. Unrolling yields a finite, quantifier-light query at the cost of bounding the explored depth: a `sat` model under depth-N unrolling is a genuine witness reachable in $\leq N$ recursive steps, which is exactly the sense in which our completeness is *bounded*. Inputs whose shortest path needs more than N steps return the terminator and are invisible at that bound.

Model decoding and witness hygiene. A `sat` model is decoded back into runnable source by a printer that extracts only `input_0`, inlines Z3's `let`-shared subterms, and maps the mangled datatype constructors back to AST nodes. Two practical points: (i) solver-chosen identifiers can be degenerate (empty or non-identifier strings) and are canonicalized to fresh valid names via a stable map—sound because the marker that fires does not depend on the spelling of a variable; and (ii) bit-vector literals are decoded as two's-complement *signed* integers (e.g. `#xffff...ff` $\to -1$), matching the signed `I64` semantics of the interpreter. The printer fully parenthesizes operators so the output round-trips through the parser.

The loop. Figure 3 shows the resulting loop for one target: build the query, solve (with k-unrolling), decode on `sat`, replay through the concrete oracle, and accept only if the marker fires. An accepted witness is then run against an independent implementation; a divergence in observable behaviour is a conformance bug.

7 Case studies

7.1 IMP: closing the loop end-to-end

IMP is the canonical small imperative "while" language of Winskel [26]. Its abstract syntax fits on a page, so a reviewer can hold the runnable specification against the textbook rules. We add one operator beyond Winskel, integer division, precisely so that division by zero is a markable condition, and we deviate in one way: reading an uninitialized variable is an error rather than defaulting to 0. These two deviations create the two markers. Arithmetic expressions, boolean expressions, and commands are three `#[smt_type]` enums with `Cloak<...>` at every recursive position; the store is `Array<String, I64>`. Big-step evaluation is three mutually recursive `#[smt_fn]`s (`eval_aexp`, `eval_bexp`, and `eval_com_inner`, wrapped by the store-fixing `eval_com`). IMP uses no quantifiers.

The loop closes (§8): synthesis returns witnesses for both markers, the printer renders them as `.imp` source, and replaying each through the concrete `eval_com` fires the same marker. For the division-by-zero marker the synthesized program is `A := (0 / 0)`; for the undefined-variable marker it is `A := v0`, an assignment that reads a variable never written.

7.2 TOML: reaching the solver frontier

TOML 1.1.0 [19] is a real-world configuration-language specification. The case study is a *complete runnable specification*: every grammar rule is realized as a `#[smt_fn]` over a code-point sequence (`Seq<U32>`). The single entry point `parse_toml(input: Seq<U32>) -> ParseResult<Value>` wraps the input in an internal parser state (cursor plus context bookkeeping), so, by Principle 1, the only free input is the document. The parser is large (on the order of 150 `#[smt_fn]`s, dozens directly self-recursive) and exercises a wide combination of theories in a *single* query: bit-vectors at two widths (`U32` code points, `I64` integer values with explicit overflow guards), the Strings theory (heavy concatenation and code/character conversions), sequences (the input and the context's nested name lists), the theory of arrays (tables as `Array<String, Value>`), unbounded integers (the cursor), and one use of Hilbert choice under a universal quantifier (to merge table keys in canonical order). Values nest mutually: arrays contain values and inline tables contain key—values that contain values, so the value layer is one large recursive SCC. The parser places markers on the many parse-error and specification-violation branches.

This combination is exactly what stresses the solver. We treat TOML not as a "TOML problem" but as a stress case that locates a general frontier (§8).

8 Evaluation and discussion

Setup. We measured on the development machine (Apple Silicon, macOS; Z3 4.15.4), synthesizing for both IMP markers with both backends, under two bounding modes: native recursion (`k=0`, Z3's `define-funs-rec`) and depth-3 unrolling (`k=3`). Every reported witness is replayed through the concrete interpreter. The IMP numbers are *measured* and reproducible. Aggregate TOML numbers are marked `[TODO: measure]`: a full sweep of the TOML targets is a multi-hour run (each target may run to a ten-minute timeout) and was not re-executed for this paper, so we report the qualitative outcome rather than quoting counts we did not reproduce here.

IMP: the loop closes (native recursion). Under native recursion (`k=0`), both backends solve both markers and every witness replays—re-executing the rendered program through the concrete `eval_com` fires the targeted marker (Table 1; the witnesses are committed as `lang/imp/input/_wit_*.imp`). Solve times are 0.4—4.9 s. Both witnesses are independently informative. The division-by-zero witness `A := (0 / 0)` reproduces the minimal path through the guard of Listing 1, and was assembled by the solver from the `Aexp` datatype constructors alone—no IMP fragment was supplied. The undefined-variable witness `A := v0` is more telling: the variable name `v0` is solver-chosen (canonicalized from a degenerate identifier returned in the model, see §6), and the assignment was constructed so that the right-hand side reads a variable the empty store never wrote. The two backends search independently—the API backend returns the deeper `v0 := ((((v0 - 1) - 1) - 1) - 1)` for the same marker—and agree on every verdict, returning *sat* for both markers.

Candidate models, and why replay matters (unrolling). Depth bounding is a delicate knob. At `k=3` the API backend returned the *same* program for both targets—`if false then skip else ((skip ; skip) ; skip)`—in two independent runs (1.1—1.2 s each). That program reaches *neither* marker: replaying it completes normally and fires nothing, so the replay check rejects it. This is the candidate-model phenomenon of §5 observed concretely—a

Table 1: IMP synthesis at `k=0` (native recursion; measured, Z3 4.15.4). "Replay" re-executes the rendered `.imp` through the concrete `eval_com`; ✓ means the targeted marker fires.

Marker	Backend	Witness (rendered `.imp`)	Time	Replay
division by zero	text (`z3_chc`)	`A := (0 / 0)`	409 ms	✓
division by zero	API (`z3_api`)	`v0 := (0 / 0)`	1625 ms	✓
undefined variable	text (`z3_chc`)	`A := v0`	834 ms	✓
undefined variable	API (`z3_api`)	`v0 := ((((v0 - 1) - 1) - 1) - 1)`	4930 ms	✓

depth-bounded unrolling can satisfy the marker assertion through the depth-cutoff terminator rather than through a genuine path, and replay is what tells a real witness from a spurious one. Here native recursion found genuine witnesses where naïve depth-3 unrolling did not, a reminder that the unroll depth must be chosen with care and that the replay safeguard is not optional.

TOML: the solver frontier. Lifting the TOML parser produces SMT that Z3 accepts without error, but the per-target queries frequently exceed the solving budget. The bottleneck is *solver capability on deeply recursive queries that combine string, integer, bit-vector, and array reasoning*—not anything specific to TOML. The mechanisms are well known: Z3's `define-funs-rec` is interpreted via recursive function unfolding rather than as a decidable theory, and quantifier reasoning (where the TOML parser uses it for canonical-order key merging) relies on model-based quantifier instantiation and e-matching—both incomplete heuristics in the general case [7, 2]. Two observations sharpen this. First, difficulty scales with recursion depth and theory combination, not with surface grammar size: shallow markers near the top of the parse are the ones most likely to be solved, while markers behind deep recursion over the input sequence are the ones that time out. Second, more unrolling does not help: it enlarges the formula without unblocking the solver, and fewer unrollings help less than one would hope. This is the same wall reported elsewhere for solver-aided reasoning over recursive, multi-theory, quantified formulas; TOML simply crosses it. We therefore present the frontier as a property of *the method×the solver*, illustrated by TOML, and we deliberately do not headline a success-rate number we did not re-measure here `[TODO: measure full TOML sweep]`.

Cross-backend agreement. On IMP the text and API backends agree on every verdict (`sat` for both markers) while exploring different witnesses, exercising the consistency check discussed in §5. The mode is cheap and catches whole-pipeline regressions; it is silent on Definition 1, which is the contract's job.

9 Related work

Solver-aided host languages. Rosette [22, 23] is the closest work: a solver-aided host language whose symbolic virtual machine compiles a host subset to constraints, stripping non-compilable constructs by concrete evaluation. RuSMT's mechanism is kindred. The distinctions are of *emphasis and discipline*, not of kind. Rosette composes a symbolic VM on top of host-language operations whose agreement with their symbolic counterparts is itself the library author's obligation; a later mechanized account of the symbolic semantics [18] closed the gap at the VM level but the per-primitive agreement remains a library-author commitment. RuSMT addresses the same obligation differently: rather than a verified VM, it makes the per-construct agreement (Definition 1) the *publicly committed* contract of the standard library, documents every primitive's Z3 formula in its doc-comment, pins both sides with example-based tests, and—for selected error-prone primitives—discharges the universal-equivalence obligation by Z3

negation proofs. The contribution is *the contract and its documented validation*, not a new symbolic engine. We do not claim Rust-as-host as a contribution.

Semantics-guided synthesis. SemGuS [13] synthesizes from syntax plus semantics, with semantics authored as constrained Horn clauses. RuSMT's users write ordinary recursive functions instead of relational CHCs; the difference is the *authoring surface* (direct-style vs. relational), not the back-end search. Synantic [14] runs in the opposite direction: it *synthesizes* a language's semantics (as syntax-directed CHCs) from a black-box interpreter via CEGIS, recovering semantics from observed behaviour, whereas RuSMT compiles from the interpreter's *source* structure. The two are complementary.

Relational interpreters. Relational interpreters in miniKanren run "backward" to synthesize programs from outputs [4, 5], but require the interpreter to be written relationally. RuSMT runs an ordinary functional interpreter forward under a solver and does not require a relational rewrite.

Executable-semantics frameworks. The K framework [20], PLT Redex [8], and Ott/Lem [21, 16] build semantics from rewrite or inference rules and derive a *suite* of tools. RuSMT uses ordinary functions and derives *one* tool—marker-directed synthesis—rather than a suite.

Bounded model finding. Alloy/Kodkod [11, 24] find bounded models of *hand-written* relational formulas. RuSMT derives the solver query from the executable semantics rather than asking the author for a formula.

Symbolic execution. KLEE [6] and symbolic execution generally explore a program's paths under a solver. RuSMT is symbolic execution of an *interpreter*, with the object-language AST as the symbolic object; path explosion is a shared concern, here mitigated (and bounded) by k-unrolling.

Interpreters as symbolic compilers. The Futamura projections [9] and the symbolic-interpreter-as-compiler line [25] observe that an interpreter can become a (symbolic) compiler by specialization/staging. RuSMT shares this conceptual kinship: lifting the interpreter *is* compiling it to a symbolic form. We do not stage; we translate the restricted source directly.

CHC verifiers. SeaHorn [10], RustHorn [15], and JayHorn [12] translate programs to CHCs for *verification* (the unknowns are predicates). RuSMT performs *synthesis* (the unknown is a program/input). RustHorn is also Rust-facing, but its target and question differ.

Conformance-test generation. Coverage- and grammar-driven test generators such as Csmith [27], and differential or fuzzing-based engine testers [17, 28], are the *application domain* here, not the contribution. RuSMT derives *targeted* witnesses from a formal executable specification rather than by coverage-driven fuzzing, and can in principle report *bounded absence* via an `unsat` result.

Summary positioning. Table 2 situates RuSMT. It occupies the integration point between solver-aided languages and semantics-directed synthesis, specialized to direct-style reference interpreters. Its value is the unified authoring surface, the soundness contract, and the empirical characterization of where the solver frontier lies—not a new theorem.

Table 2: Positioning. "Authoring" is how the semantics/spec is written; "derived query" is whether the solver query is derived from that artifact; "unknown" is what the search solves for.

System	Authoring	Derived query?	Search unknown
RuSMT	functional interpreter (Rust)	yes	input/program
Rosette	host-language program	yes	holes/inputs
SemGuS	semantics as CHCs	yes	program (grammar)
miniKanren	relational interpreter	yes	program
K / Redex	rewrite/inference rules	partial	(tool suite)
Alloy / Kodkod	relational formula (by hand)	no	relational model
KLEE	the program under test	yes	path inputs

10 Limitations and threats to validity

Restricted subset. The DSL excludes loops, mutation, and references. Some semantics are awkward to express purely recursively, and the author must learn which Rust the macros accept. This is the price of a clean SMT translation.

No general completeness. We claim only bounded completeness, and even that is contingent on the solver (§5). Beyond the unroll depth, a reachable marker may be reported unreachable; the depth is a knob, not a guarantee.

The solver frontier. The dominant limitation is solver capability on deeply recursive, multi-theory, quantified queries (§8). This is the method's frontier, not a case-study defect, and pushing it (better encodings, theory choices, abstraction, portfolio solving) is the main open problem.

Candidate models, parser, printer. Quantified and depth-unrolled queries can yield `sat` with `(:reason-unknown)` (§8). We rely on replay to reject spurious candidates; replay is therefore part of the trusted loop. Its absence would admit unsound witnesses, and so would a bug in the concrete source parser or the model-decoding pretty-printer—both of which are in the TCB (§5). The pretty-printer is the bridge from Z3 syntax back to runnable code, and its correctness is currently established by example round-trip tests rather than by a verified translation.

Soundness validation. The per-construct contract is validated by documented formulas, example-based tests, and selected manual equivalence proofs—*not* by a committed, exhaustive differential harness (§5). A primitive whose Rust and Z3 sides disagree on an untested input would silently break soundness. Building and committing such a harness is concrete future work.

Scope of the empirical claims. The IMP numbers are measured on one machine and one solver version; the TOML aggregate is not re-measured here and is marked accordingly. We make no cross-machine or cross-solver claims.

Future work. Beyond pushing the solver frontier and committing a differential audit, we are interested in a bounded-completeness theorem relating unroll depth to reachable markers, in richer case studies, and—as a longer-term *aspiration* that requires substantial further work, not a claim—in unifying algorithmic and relational specifications and supporting must/may/forbidden (nondeterministic or underspecified) conformance queries.

11 Conclusion

RuSMT lets an author write a language's reference semantics once, as an ordinary recursive Rust program, and have it serve simultaneously as the conformance oracle and—via automatic compilation to Z3 and marker-directed search—as a synthesizer of targeted test programs. The engineering that makes this trustworthy is a per-construct soundness contract, a guard discipline, a single-free-input methodology, and replay-checked synthesis soundness; the honesty is in declining to claim completeness or novelty, and in locating the real limit on the solver side. IMP shows the loop closing end-to-end; TOML shows where the frontier is. We hope the integration and the contract are useful templates for solver-aided conformance testing from executable semantics.

References

[1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design (FMCAD 2013)*, pages 1–8. IEEE, 2013.

[2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The SMT-LIB standard: Version 2.6. Technical report, Department of Computer Science, The University of Iowa, 2017. Available at `https://smt-lib.org`.

[3] Nikolaj Bjørner, Arie Gurfinkel, Kenneth L. McMillan, and Andrey Rybalchenko. Horn clause solvers for program verification. In *Fields of Logic and Computation II*, volume 9300 of *Lecture Notes in Computer Science*, pages 24–51. Springer, 2015.

[4] William E. Byrd, Michael Ballantyne, Greg Rosenblatt, and Matthew Might. A unified approach to solving seven programming problems (functional pearl). *Proceedings of the ACM on Programming Languages*, 1(ICFP):1–26, 2017.

[5] William E. Byrd, Eric Holk, and Daniel P. Friedman. miniKanren, live and untagged: Quine generation via relational interpreters (programming pearl). In *Proceedings of the 2012 Annual Workshop on Scheme and Functional Programming (Scheme '12)*, pages 8–29. ACM, 2012.

[6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI 2008)*, pages 209–224. USENIX Association, 2008.

[7] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.

[8] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. *Semantics Engineering with PLT Redex*. MIT Press, 2009.

[9] Yoshihiko Futamura. Partial evaluation of computation process—an approach to a compiler-compiler. *Higher-Order and Symbolic Computation*, 12(4):381–391, 1999. Reprint of the 1971 original.

[10] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. The SeaHorn verification framework. In *Computer Aided Verification (CAV 2015)*, volume 9206 of *Lecture Notes in Computer Science*, pages 343–361. Springer, 2015.

[11] Daniel Jackson. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 11(2):256–290, 2002.

[12] Temesghen Kahsai, Philipp Rümmer, Huascar Sanchez, and Martin Schäf. JayHorn: A framework for verifying Java programs. In *Computer Aided Verification (CAV 2016)*, volume 9779 of *Lecture Notes in Computer Science*, pages 352–358. Springer, 2016.

[13] Jinwoo Kim, Qinheping Hu, Loris D'Antoni, and Thomas W. Reps. Semantics-guided synthesis. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–32, 2021.

[14] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith J. C. Johnson, Thomas W. Reps, and Loris D'Antoni. Synthesizing formal semantics from executable interpreters. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2):362–388, 2024.

[15] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. RustHorn: CHC-based verification for Rust programs. In *Programming Languages and Systems (ESOP 2020)*, volume 12075 of *Lecture Notes in Computer Science*, pages 484–514. Springer, 2020.

[16] Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. Lem: Reusable engineering of real-world semantics. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*, pages 175–188. ACM, 2014.

[17] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. JEST: N+1-version differential testing of both JavaScript engines and specification. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE 2021)*, pages 13–24. IEEE, 2021.

[18] Sorawee Porncharoenwase, Luke Nelson, Xi Wang, and Emina Torlak. A formal foundation for symbolic evaluation with merging. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–28, 2022.

[19] Tom Preston-Werner et al. TOML: Tom's obvious, minimal language, version 1.1.0. `https://toml.io/en/v1.1.0`, 2025. Language specification.

[20] Grigore Roşu and Traian Florin Şerbănuţă. An overview of the K semantic framework. *The Journal of Logic and Algebraic Programming*, 79(6):397–434, 2010.

[21] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Tom Ridge, Susmit Sarkar, and Rok Strniša. Ott: Effective tool support for the working semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*, pages 1–12. ACM, 2007.

[22] Emina Torlak and Rastislav Bodík. Growing solver-aided languages with Rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward! 2013)*, pages 135–152. ACM, 2013.

[23] Emina Torlak and Rastislav Bodík. A lightweight symbolic virtual machine for solver-aided host languages. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*, pages 530–541. ACM, 2014.

[24] Emina Torlak and Daniel Jackson. Kodkod: A relational model finder. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2007)*, volume 4424 of *Lecture Notes in Computer Science*, pages 632–647. Springer, 2007.

[25] Guannan Wei, Oliver Bračevac, Shangyin Tan, and Tiark Rompf. Compiling symbolic execution with staging and algebraic effects. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–33, 2020.

[26] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. Foundations of Computing. MIT Press, 1993.

[27] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*, pages 283–294. ACM, 2011.

[28] Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Xiaoyang Sun, Lizhong Bian, Haibo Wang, and Zheng Wang. Automated conformance testing for JavaScript engines via deep compiler fuzzing. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2021)*, pages 435–450. ACM, 2021.